

# Job Finder (Placement Portal Application) – MAD-2 Project

## 1. Student Details

**Name:** Hari Pratap Maurya

**Roll Number:** 22f3001353

**Student Email Id:** [22f3001353@ds.study.iitm.ac.in](mailto:22f3001353@ds.study.iitm.ac.in)

I am pursuing BS in Data Science from IIT Madras and currently in diploma level.

## 2. Project Details

**Project Title:** JobFinder

This project belongs to Modern Application Development -II course, which is the part of IITM BS data science program. The Job Finder (Placement Portal Application) is a web-based system which automates campus recruitment within the institute. It will manage the interaction between Admin, Companies, and student through role-based access. This project is built using flask as backend framework and vue.js for frontend interface. This project will allow:

Admin is super user who can manage the company and students.

Companies can register themselves, post the jobs, and track the applied student applications and update their status.

Student can see all registered companies and their jobs posting and apply for different jobs based on eligibility criteria, and check the status of applications and history of applied jobs.

## 3. AI/LLM Declaration

I used ChatGPT to get suggestions in model, backend logic, some Vue component, creating API documentation and improving variable naming consistency.

The extent of AI/LLM usage is around 12-15%, limited to code suggestions and some logic building.

All the final implementation logic, debugging and integration were done manually.

## 4. Technology Used

- **Flask:** - It is used to build the server-side logic of the application.
- **Flask\_Sqlalchemy:** - It provides a straightforward way to work with databases using Python objects.
- **Jinja2:** - It is used to render dynamic HTML pages from the flask backend.
- **SQLite:** - It is used as the database to store the data.
- **Celery:** - Background tasks and reminders
- **Redis:** - Cache layer for frequently accessed data
- **Vue.JS:** - It provides the structure of page UI.
- **CSS:** - It is used for styling and improving the visual appearance.
- **Bootstrap:** - It provides quick building responsive UI.

## 5. DB Schema Design

There are total 5 tables. It includes features like role-based access of Admin, Company, and Student and timestamps for tracking records. Foreign key and Enum ensure data integrity and role management. All the relationships are properly defined with foreign keys.

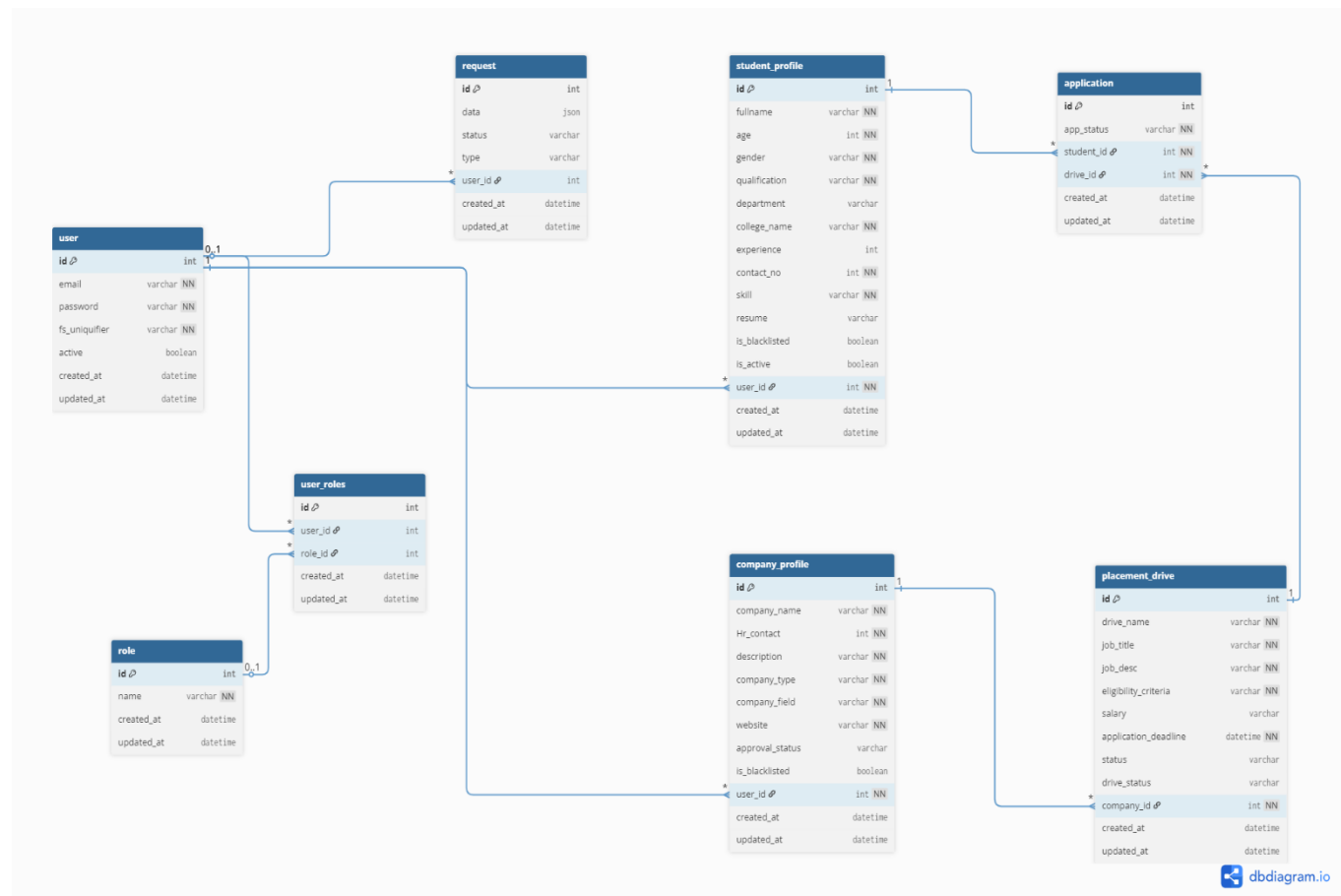
Table used in Database:

- User
- Student\_Profile
- Company\_Profile,
- Placement\_Drive
- Application
- Request
- Role
- User\_roles

### Relationship:

- **One -to-One:** User -> Student\_Profile, User -> Company\_Profile,
- **One-to-Many:** Student\_Profile -> Application, Company\_Profile -> Placement\_Drive, Placement -> Application, User -> Request
- **Many-to-Many:** Student\_Profile <-> Placement\_Drive (through Application), User <-> Roles

### ER Diagram



## 6. API Resource Endpoint

### User Management

| Endpoint               | Method | Description                         |
|------------------------|--------|-------------------------------------|
| /api/user              | GET    | Retrieve list of all users          |
| /api/user              | POST   | Create a new user                   |
| /api/user/<id>         | GET    | Retrieve details of a specific user |
| /api/user/<id>         | PUT    | Update user information             |
| /api/user/<id>         | DELETE | Delete a user                       |
| /api/user/<id>/approve | PATCH  | Approve a user account              |

### Student APIs

| Endpoint                  | Method | Description                                   |
|---------------------------|--------|-----------------------------------------------|
| /api/student/dashboard    | GET    | Fetch student dashboard data (drives, status) |
| /api/student/company/<id> | GET    | View details of a specific company            |
| /api/student/history      | GET    | Get student application history               |
| /api/student/export       | POST   | Trigger export of student data as CSV         |
| /api/student/download     | GET    | Download generated CSV file                   |
| /api/student/<id>/block   | PATCH  | Block a student account                       |
| /api/resume/<filename>    | GET    | Download/view student resume                  |

### Company APIs

| Endpoint                            | Method | Description                         |
|-------------------------------------|--------|-------------------------------------|
| /api/company/dashboard              | GET    | Fetch company dashboard data        |
| /api/company/<id>/approve           | PATCH  | Approve company registration        |
| /api/company/<id>/reject            | PATCH  | Reject company registration         |
| /api/company/<id>/block             | PATCH  | Block a company account             |
| /api/company/complete_drive/<id>    | POST   | Mark a placement drive as completed |
| /api/company/drive/<id>/application | GET    | Get all applications for a drive    |

### Placement Drive APIs

| Endpoint        | Method | Description                          |
|-----------------|--------|--------------------------------------|
| /api/drive      | GET    | Retrieve all placement drives        |
| /api/drive      | POST   | Create a new placement drive         |
| /api/drive/<id> | GET    | Retrieve details of a specific drive |
| /api/drive/<id> | PUT    | Update a placement drive             |
| /api/drive/<id> | DELETE | Delete a placement drive             |

## Application APIs

| Endpoint              | Method | Description                     |
|-----------------------|--------|---------------------------------|
| /api/application      | GET    | Retrieve all applications       |
| /api/application      | POST   | Apply to a placement drive      |
| /api/application/<id> | GET    | Retrieve a specific application |
| /api/application/<id> | PUT    | Update application status       |
| /api/application/<id> | DELETE | Delete an application           |

## Admin APIs

| Endpoint             | Method | Description                        |
|----------------------|--------|------------------------------------|
| /api/admin/dashboard | GET    | Fetch admin dashboard statistics   |
| /api/admin/search    | GET    | Search users, companies, or drives |

## Request Management APIs

| Endpoint                  | Method | Description               |
|---------------------------|--------|---------------------------|
| /api/request/<id>/approve | POST   | Approve a pending request |
| /api/request/<id>/reject  | POST   | Reject a pending request  |

## 6. Architecture and Features

The project follows a client-server architecture. The Flask backend manages APIs, authentication, and database operations, while the Vue.js frontend provides the interactive user interface. Backend structure includes modules for models, routes, tasks. The frontend uses Vue components and Vue Router for navigation. **Implemented Features:** - User authentication with role-based access - Admin dashboard for managing users and drives - Placement drive creation and management - Student application submission and tracking - Background jobs for reminders and reports

### Implemented Features:

- User login (Admin, Company, Student) and Registration (Company, Student) and cannot register with same email id if register it before.
- Secure password hashing.
- role-based access.
- Student can not apply again in same drive if applied before.
- Admin is superuser and see the total number of register company, placement drive, student, applications, and approve the company and drive, block and unblock the student and company, see the ongoing drives and student application and their resume.

- Student can apply in many drives, check the status, Upload resume.
- Company can create drives, Edit, Delete and Mark as complete after approval by admin, track and update the student application.
- Background task like send mail for application deadline, send monthly report to admin in csv format and student can export student history in csv format.

## **7. Video and GitHub**

Demo video: -

[https://drive.google.com/file/d/1MTVA2oa2A0HMOj\\_Lf6JBK8OyqYaM2oep/view?usp=drive\\_link](https://drive.google.com/file/d/1MTVA2oa2A0HMOj_Lf6JBK8OyqYaM2oep/view?usp=drive_link)

Project GitHub: -

<https://github.com/22f3001353/jobfinder-mad2>